CC232 - Algoritmos y Estructuras de Datos
Examen Parcial - Semanas 1 a 7

Modalidad: individual, presencial, lápiz y papel. **Puntaje:** 20 puntos. **Tiempo de resolución estimado:** 150 minutos. **Tiempo máximo de sesión:** 180 minutos.

Nombre y apellidos: _____ **Código:** _____

Indicaciones generales

1. Resuelve todos los incisos. Cuando se soliciten estados, indica valores, índices, referencias y contadores suficientes para comprobarlos. Un resultado final sin traza no sustituye la traza solicitada.
2. Cuando se solicite un árbol, dibuja sus nodos y enlaces a partir de los datos proporcionados. Identifica claramente raíz, hijos y los campos pedidos. El enunciado no proporciona representaciones gráficas resueltas.
3. Escribe Java simplificado únicamente donde se pida código. Mantén las firmas indicadas, usa solo los campos y operaciones del curso y explica por qué se conservan los invariantes. No utilices Java Collections ni bibliotecas que implementen las estructuras evaluadas.
4. Justifica las complejidades mediante el trabajo efectuado. Distingue localización de modificación y costo individual de costo amortizado. Escribir únicamente `O(1)`, `O(h)` u `O(log n)` no constituye justificación.
5. Usa índices desde cero. En BST simple, **h** representa la altura expresada en aristas. En AVL, `height(null)=0`, una hoja tiene `height=1` y `balanceFactor(u)=height(u.left)-height(u.right)`. En `BinaryHeap` se utiliza un **min-heap**, con `left(i)=2i+1`, `right(i)=2i+2` y `parent(i)=(i-1)/2` para `i>0`.
6. No utilices celulares, computadoras, Internet, asistentes de IA, apuntes ni material externo durante el examen. En los incisos de implementación, los errores menores de sintaxis se distinguirán de los errores de lógica y de invariantes.

Distribución orientativa: problema 1, 22 min y 3 puntos; problema 2, 26 min y 3 puntos; problema 3, 37 min y 5 puntos; problema 4, 33 min y 4 puntos; problema 5, 32 min y 5 puntos. Total: **150 min, 20 puntos**.

Problema 1. Edición transaccional de una bitácora en memoria - 3 puntos

Una plataforma industrial registra eventos ordenados. El editor mantiene la misma secuencia lógica mediante un arreglo dinámico o una lista doblemente enlazada circular con centinela `dummy`. Su estado inicial es:

- Secuencia lógica: `[12, 24, 36, 48, 60, 72, 84]`.
- Arreglo: `a.length=8`, `n=7`, `currentIndex=2`.
- `DLList`: `n=7`, `current` referencia el nodo de valor 36, y `dummy.next` y `dummy.prev` referencian los extremos.

Se ejecutan, estrictamente en este orden: `insertAfterCurrent(42)`; `current` avanza dos nodos o posiciones; `removeCurrent()`; `insertAfterCurrent(66)`; `get(5)`. Cada inserción deja `current` en el mismo elemento. Después de una eliminación, `current` pasa al sucesor, salvo que se elimine el último nodo, caso en el que pasa al predecesor; si la lista queda vacía, pasa a `dummy`.

a) [1,00] Traza las cinco operaciones para ambas representaciones. Escribe la secuencia lógica después de cada modificación, el estado final del arreglo (`a[0..n-1]`, `n`, capacidad y `currentIndex`), la referencia final de `current` en `DLList` y el resultado de `get(5)`. No confundas el índice del arreglo con una referencia a nodo.

b) [0,75] Cuenta los elementos desplazados en cada modificación del arreglo, las asignaciones a campos `prev` y `next` en cada modificación de `DLList` y el número de enlaces recorridos para `get(5)` si `getNode` empieza desde el extremo más cercano. No cuentes la creación de nodos ni las actualizaciones de `n` como asignaciones de enlace. Separa localización de modificación.

c) [0,75] Escribe el cuerpo de `void removeCurrent()` para `DLList`, suponiendo `class Node { int x; Node prev, next; }` y los campos de lista `Node current`, `Node dummy` e `int n`. Supón que la operación se llama solo cuando `n>0` y `current!=dummy`. Conserva las referencias necesarias antes de desconectar el nodo y aplica la regla de cursor establecida.

d) [0,50] Formula un invariante del arreglo y dos de la lista circular que permitan detectar una pérdida de elementos o enlaces tras la traza. Decide qué representación conviene si el 85 % de las operaciones se realiza alrededor de `current` y cuál si el 85 % corresponde a `get(i)` con índices arbitrarios. Fundamenta la selección en los costos de este caso.

Problema 2. Despacho de tareas con FIFO, rotación y registro LIFO - 3 puntos

Un sistema de ejecución utiliza `ArrayQueue ready` para turnos FIFO y `LinkedStack finished` para registrar tareas terminadas. En la cola, `a.length=8`, `j=5`, `n=5` y las únicas posiciones ocupadas son `a[5]=T1`, `a[6]=T2`, `a[7]=T3`, `a[0]=T4`, `a[1]=T5`. Las otras posiciones son `null`. La pila empieza con `head=null` y `n=0`.

Los eventos, en orden, son: (1) `T1` sale de `ready` y termina, por lo que se apila en `finished`; (2) `T2` sale de `ready` y se reinserta al final porque no termina; (3) llega `T6`; (4) llega `T7`; (5) llega `T8`; (6) llega `T9`; (7) llega `T10`. El último `add` exige ampliar la capacidad. El crecimiento copia los `n` elementos a un arreglo de capacidad `2*n`, reinicia `j=0` y posteriormente inserta el nuevo elemento. Para el evento (2), realiza primero `remove()` y después `add()`.

a) [1,25] Traza los siete eventos. Para cada uno, registra `j`, `n`, la secuencia FIFO y cada posición física escrita o liberada. Escribe además tres estados completos y distintos: inmediatamente antes del `resize`, inmediatamente después del `resize`, e inmediatamente después de insertar `T10`. Incluye el contenido final de `finished` desde `head`.

b) [0,75] Escribe el cuerpo correcto de `void resize()` para `ArrayQueue` utilizando `a`, `j` y `n`. La copia debe preservar FIFO aunque el almacenamiento esté envuelto alrededor del final físico del arreglo. Especifica el índice físico de origen del elemento lógico `k`.

c) [0,50] Si un programador sustituye la copia correcta por `b[k]=a[k]` y luego fija `j=0`, identifica la primera tarea que ocuparía incorrectamente el frente **a partir del estado real previo al crecimiento**. Explica qué tarea debería encabezar la cola y por qué la versión defectuosa cambia el comportamiento observable.

d) [0,50] Escribe `void push(T x)` para la `LinkedStack` usando un nodo con campos `x` y `next`. Justifica el invariante de `head`, el nuevo valor de `n` y el costo de `push`. Indica además el costo amortizado de `ArrayQueue.add`, el peor caso de una llamada que ejecuta `resize` y la razón de la diferencia.

Problema 3. Índice de activos: eliminación segura en un BST - 5 puntos

Un servicio de inventario mantiene identificadores únicos mediante un **BST simple no balanceado**, con campos `root`, `n`, y nodos `x`, `left`, `right`, `parent`. Inserta en este orden, en un árbol vacío:

`64, 32, 88, 20, 49, 76, 96, 40, 57, 70, 82, 73`.

La implementación de `remove(x)` rechaza claves ausentes. Cuando elimina un nodo con dos hijos, **copia la clave del sucesor inorder en el nodo original** y elimina físicamente el nodo sucesor mediante `splice`. El método `splice(Node u)` presupone que `u` tiene, como máximo, un hijo y **no modifica** `n`; `remove` decrementa `n` exactamente una vez en una eliminación exitosa.

a) [1,00] Dibuja el BST inicial completo. Señala las relaciones `parent` de los nodos con claves `70`, `73` y `76`. Escribe el recorrido inorder inicial y el valor inicial de `n`. No supongas balanceo automático.

b) [1,50] Ejecuta `remove(64)`. Traza la búsqueda del sucesor desde el subárbol derecho, identifica qué hijo, si lo hay, conserva ese sucesor, y enumera todas las relaciones padre-hijo modificadas. Dibuja el árbol final y explica por qué ningún subárbol deja de ser alcanzable desde `root`.

c) [0,75] A partir del resultado anterior, ejecuta `remove(20)` y después `remove(21)`. Distingue claramente el resultado booleano de cada llamada, los tamaños después de cada operación y el inorder final. Explica por qué una búsqueda fallida que retorna un último nodo visitado no autoriza a eliminarlo.

d) [1,25] Escribe el cuerpo de `void splice(Node u)`. Debe cubrir eliminación de raíz, nodo hoja y nodo con un único hijo, reconectar correctamente `parent` y preservar todos los nodos supervivientes. Explica por qué el método no ejecuta `n--`.

e) [0,50] Justifica los costos de `findLast`, `contains`, `add` y `remove` en términos de la altura `h` (un camino de `h` aristas visita hasta `h+1` nodos), separando búsqueda y modificación. Indica cómo cambia la cota en un BST degenerado y por qué no corresponde afirmar siempre `O(log n)`.

Problema 4. Índice AVL para consultas de baja latencia - 4 puntos

Una empresa necesita mantener búsquedas rápidas en un índice dinámico. Su AVL se implementa con `Node { int x; int height; Node left; Node right; }`, sin `parent`, y utiliza los métodos `height`, `updateHeight`, `balanceFactor`, `rotateLeft` y `rotateRight` de la Semana 6. Las rotaciones ya actualizan las alturas afectadas y retornan la nueva raíz local. La inserción usa recursión, actualiza alturas al regresar y llama a `rebalance` en cada nodo del camino de retorno; las claves duplicadas no se insertan.

Partiendo de un AVL vacío, inserta en este orden: `30, 100, 10, 15, 120, 70, 95, 60`. Después ejecuta `add(90)`.

a) [1,00] Dibuja el AVL tras las primeras ocho inserciones y anota `height` y `balanceFactor` de cada nodo. Comprueba con valores concretos que hasta ese momento no es necesaria ninguna rotación.

b) [1,50] Traza `add(90)`. Identifica el primer nodo que queda desbalanceado en el retorno recursivo, calcula su factor de balance y el de su hijo relevante, clasifica el caso LL, RR, LR o RL y especifica en orden las rotaciones necesarias. Identifica expresamente el subárbol intermedio que debe conservarse. Dibuja el AVL resultante e indica las alturas finales de los nodos modificados y de la raíz.

c) [1,00] Escribe el cuerpo de `Node rebalance(Node u)` utilizando los auxiliares dados. Debe reconocer **los cuatro casos** LL, RR, LR y RL, actualizar la referencia al hijo cuando se realice una rotación doble y retornar siempre la raíz local correcta. No vuelvas a implementar las rotaciones.

d) [0,50] Explica por qué conservar el inorder no basta para certificar un AVL correcto. Identifica otros dos invariantes necesarios y justifica por qué la rotación es `O(1)` pero una inserción AVL completa cuesta `O(log n)`.

Problema 5. Centro de incidentes: construcción y operación de una cola de prioridad - 5 puntos

Un sistema de atención de incidentes recibe severidades numéricas, donde **menor número significa mayor prioridad**. Debe admitir inserciones y retirar siempre la severidad mínima. Utiliza exclusivamente el `BinaryHeap` mínimo de la Semana 7, no `java.util.PriorityQueue`. Todos los valores de este caso son distintos.

El constructor recibe el arreglo físico `values=[17, 9, 24, 5, 12, 20, 28, 3, 7, 15]`. Inicialmente `n=10`, `a.length=10`. El constructor ejecuta `heapify()` desde `i=n/2-1` hasta `0`. Después se ejecutan `add(4)`, `remove()` y `remove()`, en ese orden. En `add(4)`, si se requiere crecer, la nueva capacidad es `2*n`. Al retirar el mínimo se sustituye la raíz por el último elemento válido antes de ejecutar `trickleDown(0)`. Cuando ambos hijos son válidos se selecciona el de menor clave; el heap solo intercambia si el hijo elegido es estrictamente menor que el padre.

a) [1,50] Ejecuta `heapify()` manualmente. Para cada índice inicial de `trickleDown` procesado (`4, 3, 2, 1, 0`), anota los pares de índices intercambiados y el arreglo después de terminar esa llamada. Dibuja el árbol completo asociado al arreglo final y verifica las relaciones padre-hijo que garantizan el invariante min-heap.

b) [1,25] Traza `add(4)` y las dos llamadas sucesivas a `remove()`. En cada operación indica los índices intercambiados, el mínimo retirado cuando corresponda, `n`, `a.length` y el arreglo lógico final `a[0..n-1]`. Justifica por qué no se puede desplazar arbitrariamente un nodo del árbol a otra posición para simplificar la operación.

c) [1,50] Escribe el cuerpo de `void trickleDown(int i)` para la representación por arreglo. Verifica si existen los hijos antes de acceder a ellos, selecciona el menor hijo válido, detente cuando se cumpla la propiedad de heap y actualiza correctamente el índice tras cada intercambio. No utilices recursión, ordenamiento general ni clases de colecciones.

d) [0,75] Demuestra, mediante el número de nodos a cada altura, por qué construir con `heapify` desde abajo cuesta `O(n)` y por qué insertar los `n` valores sucesivamente tiene cota `O(n log n)`. Indica el costo de `peek`, `add` y `remove`; finalmente, explica qué requisito del centro de incidentes no garantiza una `ArrayQueue` FIFO aunque ambas estructuras almacenen las mismas tareas.

Fin del examen. Comprueba que cada respuesta contiene la representación, el algoritmo, la justificación y la complejidad solicitados. No se evalúan únicamente resultados aislados.